

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Facultad de Ingeniería

Grupo de Investigación en Multimedia Interactiva y Animación Digital — Línea de Interacción Persona-Máquina (HCI)

Estudio Experimental de Carga Cognitiva y Experiencia de Usuario en Interfaces Digitales

Presentado por:

Javier Camilo Murcia Najar — 20221020106

Jefferson David Ortiz Buitrago — 20221020110

Carmen Sofía Flórez Juajibioy — 20221020140

Laura Natalia Pardo Castillo — 20221020146

Daniel Esteban Bulla López — 20251020021

Octubre de 2026

1. INTRODUCCIÓN

El presente documento corresponde al segundo avance del proyecto de investigación “Estudio Experimental de Carga Cognitiva y Experiencia de Usuario en Interfaces Digitales”, desarrollado en el marco del Grupo de Investigación en Multimedia Interactiva y Animación Digital de la Universidad Distrital Francisco José de Caldas, dentro de la línea de Interacción Persona-Máquina (HCI).

El proyecto toma como punto de partida el Laboratorio Virtual de Redes desarrollado por el semillero ARVis, implementado mediante el motor de desarrollo Unity y dispositivos de realidad virtual Meta Quest. A partir de este prototipo, se plantea analizar cómo la complejidad de las tareas de manipulación de equipos de red influye en la carga cognitiva percibida, el desempeño operacional y la experiencia de usuario.

A diferencia del primer avance, enfocado en los fundamentos teóricos y el diseño metodológico, esta segunda entrega se concentra en la documentación técnica del entorno de desarrollo y en la preparación de los componentes necesarios para implementar los escenarios experimentales.

Para ello, se presentan los requisitos de hardware y software, los procedimientos de instalación, la ejecución del proyecto en PC y Meta Quest, la configuración de las conexiones de red, los parámetros de compilación para Android, la identificación de posibles errores, el diagrama de despliegue del sistema, la arquitectura del software, las interacciones y los scripts principales y el funcionamiento del terminal Telnet.

Esta documentación tiene como finalidad facilitar la reproducción del entorno de trabajo, establecer una configuración técnica común entre los integrantes del equipo y proporcionar una base para las siguientes fases de desarrollo y experimentación.

2. ALCANCE

El presente avance comprende la revisión y documentación técnica del Laboratorio Virtual de Redes existente, así como la identificación de los elementos necesarios para su utilización como instrumento experimental en el estudio de carga cognitiva y experiencia de usuario.

El proyecto contempla la comparación de dos escenarios virtuales desarrollados sobre una misma base tecnológica. El escenario A corresponde al prototipo existente, mientras que el escenario B constituye una extensión que incorporará una mayor complejidad en las tareas de manipulación de equipos de red.

Ambos escenarios deberán conservar condiciones técnicas comparables, de manera que las diferencias observadas en el desempeño de los participantes puedan relacionarse con la complejidad de las tareas y no con variaciones en la plataforma utilizada.

2.1. Actividades comprendidas

- Documentación del entorno de desarrollo y de la estructura técnica del prototipo existente.

- Identificación de los requisitos de hardware y software necesarios para su instalación y ejecución.
- Descripción de los procedimientos para ejecutar el proyecto desde Unity y mediante un dispositivo Meta Quest.
- Documentación de la configuración de red y de los mecanismos de comunicación entre la aplicación y el emulador de dispositivos.
- Identificación de los parámetros necesarios para compilar la aplicación en Android.
- Revisión de posibles errores de instalación, configuración y ejecución.
- Elaboración del diagrama de despliegue del sistema.
- Descripción de la arquitectura del software, de las interacciones y los scripts principales y del funcionamiento del terminal Telnet.
- Preparación de una base documental para el desarrollo del segundo escenario experimental.

2.2. Actividades previstas para las siguientes fases

El diseño general de la investigación contempla incorporar mecanismos de telemetría que permitan registrar automáticamente tiempos de ejecución, errores, intentos, solicitudes de ayuda y abandonos.

Asimismo, se prevé aplicar los instrumentos NASA-TLX y System Usability Scale (SUS) después de las pruebas experimentales, con el propósito de evaluar la carga cognitiva y la usabilidad percibida.

Estas actividades forman parte del alcance general del proyecto de investigación y no deben interpretarse como funcionalidades completamente implementadas o validadas durante el presente avance.

2.3. Limitaciones

El proyecto se desarrolla con fines académicos e investigativos, por lo que no contempla la comercialización del aplicativo ni su publicación en tiendas de aplicaciones.

Su utilización se encuentra orientada a dispositivos Meta Quest compatibles con la configuración de realidad virtual del proyecto. El modelo específico utilizado durante las pruebas deberá documentarse una vez se confirme su disponibilidad.

De igual manera, la ejecución de las sesiones experimentales dependerá de la disponibilidad de los dispositivos, la preparación de los escenarios y la validación de los mecanismos de recolección de datos.

3. REQUISITOS DEL SISTEMA

Para garantizar la instalación y ejecución del Laboratorio Virtual de Redes se identificaron los componentes de hardware y software necesarios para trabajar con el proyecto existente.

3.1. Requisitos de hardware

Componente	Requisito
Equipo de desarrollo	Computador compatible con Unity 6 y con capacidad suficiente para ejecutar el editor, importar los recursos y compilar el proyecto para Android.
Memoria RAM	Se recomienda disponer de 16 GB para el entorno de desarrollo; este valor debe validarse según el equipo utilizado.
Almacenamiento	Espacio disponible para el proyecto, las dependencias, la carpeta Library y los archivos generados durante la compilación.
Dispositivo de realidad virtual	Visor Meta Quest compatible con OpenXR y con el modo desarrollador habilitado.
Cable de conexión	Cable USB-C compatible con transferencia de datos y depuración ADB.
Red local	Conexión de red que permita la comunicación entre el computador y el dispositivo Meta Quest.
Emulador de red	Herramienta de simulación o emulación que permita establecer conexiones con las consolas de Router-1, PC1, PC2 y PC3.

El modelo exacto del visor, las especificaciones del computador y el emulador de red utilizado deberán registrarse en la versión definitiva de la documentación técnica, una vez verificados en el entorno de pruebas.

3.2. Requisitos de software

A partir de la revisión de los archivos de configuración del proyecto se identificaron los siguientes componentes:

Software o componente	Versión
Unity Editor	6000.2.13f1 (Unity 6.2)
OpenXR Plugin	1.16.0
XR Interaction Toolkit	3.2.2
XR Hands	1.7.3
Input System	1.14.2

Software o componente	Versión
Universal Render Pipeline	17.2.0
Unity UI (uGUI)	2.0.0
Android Build Support	Módulo requerido
OpenJDK	Incluido en las herramientas de Android de Unity Hub
Android SDK y NDK Tools	Incluidos en las herramientas de Android de Unity Hub
Git	Para la gestión y descarga del repositorio
Visual Studio o Rider	Entorno de programación compatible con Unity
ADB	Para instalación, depuración y transferencia de archivos

Se recomienda utilizar la misma versión de Unity en todos los equipos de desarrollo para reducir posibles incompatibilidades entre paquetes, recursos y configuraciones.

4. INSTALACIÓN DEL PROYECTO

La instalación del Laboratorio Virtual de Redes comprende la descarga del código fuente, la preparación del entorno de desarrollo, la importación de los recursos y la configuración de los servicios necesarios para su funcionamiento.

4.1. Descarga del repositorio

Inicialmente, se debe obtener una copia local del repositorio que contiene el proyecto desarrollado por el semillero ARVis.

Para ello, puede utilizarse Git mediante el siguiente comando:

```
git clone [URL DEL REPOSITORIO]
```

Posteriormente, se debe ingresar a la carpeta descargada y comprobar que contiene los directorios principales del proyecto Unity, entre ellos Assets, Packages y ProjectSettings.

La dirección del repositorio deberá incorporarse una vez se confirme la URL correspondiente al proyecto utilizado por el equipo.

4.2. Instalación y configuración de Unity

Se debe instalar Unity Hub y, desde este administrador, descargar la versión 6000.2.13f1 de Unity Editor.

Durante la instalación es necesario incluir Android Build Support y sus componentes OpenJDK y Android SDK & NDK Tools, debido a que la aplicación está destinada a ejecutarse en dispositivos Meta Quest mediante una compilación Android.

Una vez instalado el editor, se debe abrir Unity Hub, seleccionar la opción para agregar un proyecto existente e indicar la carpeta que contiene los archivos descargados.

En la primera apertura, Unity realiza la importación de recursos y la resolución de dependencias definidas en Packages/manifest.json. Este procedimiento puede tardar varios minutos, dependiendo de las características del equipo.

4.3. Configuración de realidad virtual

Después de abrir el proyecto, se debe comprobar que el sistema de realidad virtual se encuentre configurado correctamente.

Para ello, se accede a:

Edit → Project Settings → XR Plug-in Management

En la configuración correspondiente a Android se debe verificar que OpenXR esté habilitado y que se encuentren disponibles las características necesarias para la compatibilidad con Meta Quest, incluidos los perfiles de controladores y seguimiento de manos utilizados por el proyecto.

El entorno incorpora herramientas de XR Interaction Toolkit para gestionar las interacciones dentro del espacio tridimensional.

4.4. Apertura de la escena principal

La escena principal del laboratorio se encuentra en:

Assets/Scenes/LabScene.unity

Esta escena contiene el entorno virtual donde se desarrollan las actividades relacionadas con la identificación, manipulación y configuración de dispositivos de red.

Se debe abrir la escena y comprobar que los objetos y componentes se carguen correctamente antes de iniciar las pruebas de ejecución.

4.5. Configuración de red

El proyecto utiliza un archivo denominado config.json para establecer la dirección IP del equipo que ejecuta el emulador de red y los puertos correspondientes a las conexiones con los dispositivos simulados.

La estructura esperada es la siguiente:

```
{
  "ip": "192.168.X.X",
  "puerto1": 5000,
  "puerto2": 5001,
  "puerto3": 5002,
  "puerto4": 5003
}
```

La dirección IP debe reemplazarse por la dirección real del computador que ejecuta el servicio de red. Los puertos presentados son valores ilustrativos y deben coincidir con aquellos que se encuentren habilitados en el emulador.

La relación de los puertos con las consolas es la siguiente:

Parámetro	Consola
puerto1	Router-1
puerto2	PC1
puerto3	PC2
puerto4	PC3

El archivo debe ubicarse en el directorio de datos persistentes de la aplicación, determinado por Unity mediante Application.persistentDataPath.

Cuando no se encuentra este archivo, la configuración predeterminada utiliza la dirección 127.0.0.1 y los puertos 8080, 8081 y 8082, mientras que el cuarto puerto no dispone de una asignación válida.

Por esta razón, para realizar conexiones desde Meta Quest hacia el computador es necesario proporcionar una configuración explícita con una dirección IP accesible desde la red local.

5. EJECUCIÓN DEL LABORATORIO VIRTUAL

El proyecto permite utilizar el entorno desde el editor de Unity y contempla su ejecución como aplicación Android en un dispositivo Meta Quest.

5.1. Ejecución en PC

Para ejecutar el laboratorio desde el computador se debe abrir la escena LabScene en Unity Editor.

Antes de iniciar la aplicación, es necesario preparar el servicio de red que proporciona acceso a las consolas de los dispositivos simulados y comprobar que el archivo config.json contiene los parámetros adecuados.

Una vez preparada la configuración, se puede iniciar la ejecución mediante el botón Play del editor.

El proyecto incluye un sistema de control mediante teclado y ratón, implementado a través del script KeyboardVRCameraController, que facilita la interacción con el entorno sin necesidad de utilizar el visor durante determinadas tareas de desarrollo.

Los controles identificados son los siguientes:

Acción	Control
Movimiento y rotación del rig	WASD / Q-E
Rotación de cámara	Ratón
Bloqueo o liberación del cursor	ESC
Agarre izquierdo y derecho	G / H
Disparadores	Clic derecho / clic izquierdo
Movimiento de mano izquierda	Teclado numérico 4/6/8/5/7/9
Movimiento de mano derecha	J/L/I/K/U/O
Mostrar u ocultar terminal	T

Estos controles permiten revisar distintas interacciones del prototipo durante el proceso de desarrollo. No obstante, su disponibilidad en la escena principal debe confirmarse, ya que no se encontró el componente XRSimulatorController referenciado en LabScene.unity (véase la sección 10.4.2).

5.2. Ejecución en Meta Quest

Para ejecutar el laboratorio en un dispositivo Meta Quest se requiere generar previamente una compilación compatible con Android.

El dispositivo debe tener habilitado el modo desarrollador, estar conectado al computador mediante un cable USB-C de datos y contar con la autorización de depuración USB.

Desde Unity, se debe acceder a:

File → Build Profiles → Android

Posteriormente, se selecciona la plataforma Android y se utiliza la opción Switch Platform cuando corresponda.

Antes de compilar, se debe verificar que la escena LabScene esté incluida y habilitada dentro de la lista de escenas de la aplicación.

Una vez configurado el entorno, se selecciona el dispositivo en Run Device y se utiliza la opción Build And Run para generar e instalar la aplicación.

Después de la instalación, debe prepararse el archivo de configuración de red en el almacenamiento persistente del dispositivo. Mediante ADB puede utilizarse el siguiente comando, reemplazando el identificador por el nombre real del paquete:

```
adb push config.json /sdcard/Android/data/[package-name]/files/config.json
```

La ruta correspondiente debe existir previamente. Para ello, puede ser necesario ejecutar la aplicación una primera vez y comprobar la creación de su directorio de almacenamiento.

Durante la ejecución, el dispositivo Meta Quest y el computador que proporciona el servicio de red deben encontrarse conectados a una red que permita la comunicación entre ambos.

El funcionamiento completo de la aplicación en el visor deberá confirmarse mediante pruebas de instalación, visualización, interacción y conectividad.

5.3. Registro y extracción de resultados

El proyecto contiene mecanismos para registrar tiempos de ejecución asociados con determinadas pruebas del laboratorio.

Los scripts InitialTestManager y WireTestManager incorporan registros en el archivo tiempos.txt, almacenado dentro del directorio persistente de la aplicación.

Este archivo permite conservar información relacionada con la fecha, el tipo de prueba y el tiempo registrado en segundos.

Para extraer el archivo desde Meta Quest hacia el computador se puede emplear:

```
adb pull /sdcard/Android/data/[package-name]/files/tiempos.txt .
```

Este procedimiento permite recuperar los registros para su revisión posterior.

Es importante aclarar que la existencia del registro de tiempos no implica que todos los indicadores previstos para el estudio experimental se encuentren implementados. La incorporación y validación de registros de errores, intentos, solicitudes de ayuda y abandonos deberá revisarse durante el desarrollo de los escenarios.

6. COMPILACIÓN DEL PROYECTO

La compilación permite transformar el proyecto desarrollado en Unity en una aplicación Android ejecutable en dispositivos Meta Quest.

6.1. Configuración de compilación

A partir de la configuración identificada en el proyecto se registraron los siguientes parámetros:

Parámetro	Configuración
Plataforma	Android
Scripting Backend	IL2CPP
Arquitectura	ARM64
Entrada de aplicación	GameActivity
API mínima	25
API objetivo	30
Espacio de color	Linear
Plantillas Gradle personalizadas	Habilitadas
Versión de la aplicación	0.1.0
Version Code	1
Frecuencia objetivo de rendimiento VR	72 FPS
Escala de resolución de textura	0.75

Los parámetros relacionados con el rendimiento se encuentran asociados a la configuración del prototipo y no constituyen, por sí mismos, evidencia de que el dispositivo mantenga de manera estable dicha frecuencia durante las pruebas.

6.2. Verificación de escenas

Durante la revisión de los archivos del proyecto se identificó que la escena principal LabScene aparece desactivada en la configuración de compilación almacenada.

Por esta razón, antes de generar el archivo APK es necesario revisar la lista de escenas y habilitar la escena principal.

También se identificó una referencia a HelloCardboard, correspondiente a una escena que debe revisarse para determinar si continúa siendo necesaria.

Estas verificaciones son importantes para evitar generar una aplicación que inicie sin presentar correctamente el laboratorio.

6.3. Generación del APK

Después de configurar la plataforma Android, las escenas y las dependencias necesarias, se debe utilizar la opción Build o Build And Run.

La primera opción permite generar el archivo APK en una ubicación seleccionada, mientras que la segunda permite realizar la compilación y ejecutar el proceso de instalación sobre un dispositivo compatible conectado al computador.

Finalizado el procedimiento, se debe verificar que no existan errores de compilación y que la aplicación pueda iniciarse correctamente.

La compilación y ejecución efectiva deberán documentarse mediante los resultados obtenidos durante las pruebas técnicas del equipo.

7. ERRORES FRECUENTES Y POSIBLES SOLUCIONES

La revisión de la configuración y de los mecanismos de funcionamiento del prototipo permitió identificar distintos problemas que pueden presentarse durante la instalación, compilación o ejecución.

Las siguientes situaciones corresponden a errores posibles y comprobaciones recomendadas. Su inclusión no implica que todos hayan ocurrido durante las pruebas realizadas.

Problema	Posible causa	Solución recomendada
La aplicación inicia en negro o sin mostrar la escena principal.	La escena LabScene no está habilitada en la configuración de compilación.	Activar la escena principal en Build Profiles.
Aparece una advertencia relacionada con HelloCardboard.	Existe una referencia obsoleta a una escena.	Revisar y retirar la referencia cuando no sea necesaria.
La terminal no establece conexión.	Dirección IP incorrecta, archivo config.json ausente o puertos mal configurados.	Verificar el archivo, la dirección del servidor y los puertos utilizados.
La aplicación funciona en PC, pero no conecta desde Quest.	Los dispositivos se encuentran en redes diferentes, el firewall bloquea la comunicación o el servidor no escucha en una interfaz accesible.	Comprobar la red local, los puertos y la configuración del firewall.
No se selecciona correctamente un dispositivo de red.	Los nombres de las consolas no coinciden con los utilizados por el sistema.	Comprobar las denominaciones Router-1, PC1, PC2 y PC3.

Problema	Posible causa	Solución recomendada
Meta Quest no aparece como dispositivo disponible.	Modo desarrollador deshabilitado, cable sin transferencia de datos o depuración USB no autorizada.	Activar el modo desarrollador y autorizar la conexión ADB.
Falla la compilación Android.	Componentes SDK, NDK o JDK ausentes o incompatibles.	Verificar Android Build Support y las herramientas instaladas desde Unity Hub.
Los controles de realidad virtual no responden.	Configuración incorrecta de OpenXR o de los perfiles de interacción.	Revisar XR Plug-in Management y las características habilitadas.
Algunos materiales aparecen de color rosado.	Incompatibilidad entre shaders y Universal Render Pipeline.	Revisar los materiales y utilizar shaders compatibles con URP.
El proceso de importación o compilación tarda demasiado.	Cantidad elevada de recursos, dependencias o archivos de proyectos anteriores.	Revisar el contenido del repositorio y excluir archivos temporales o compilaciones innecesarias.

Adicionalmente, se identificaron recursos asociados con reconocimiento de voz mediante Vosk. Su uso efectivo debe confirmarse antes de retirar archivos, ya que eliminar recursos sin verificar sus dependencias podría afectar otras funciones del proyecto.

8. DIAGRAMA DE DESPLIEGUE

El diagrama de despliegue representa la distribución de los componentes del Laboratorio Virtual de Redes entre el computador utilizado para el desarrollo y el dispositivo Meta Quest donde se ejecuta la aplicación de realidad virtual.

El entorno se organiza en dos nodos principales.

El primero corresponde al computador, que contiene el proyecto Unity, las herramientas de compilación y el emulador o servicio de red encargado de proporcionar acceso a las consolas de los dispositivos simulados.

El segundo nodo corresponde al dispositivo Meta Quest, que ejecuta la aplicación compilada para Android y almacena tanto la configuración de conexión como los registros generados durante las pruebas.

La comunicación entre ambos nodos se realiza mediante dos mecanismos principales.

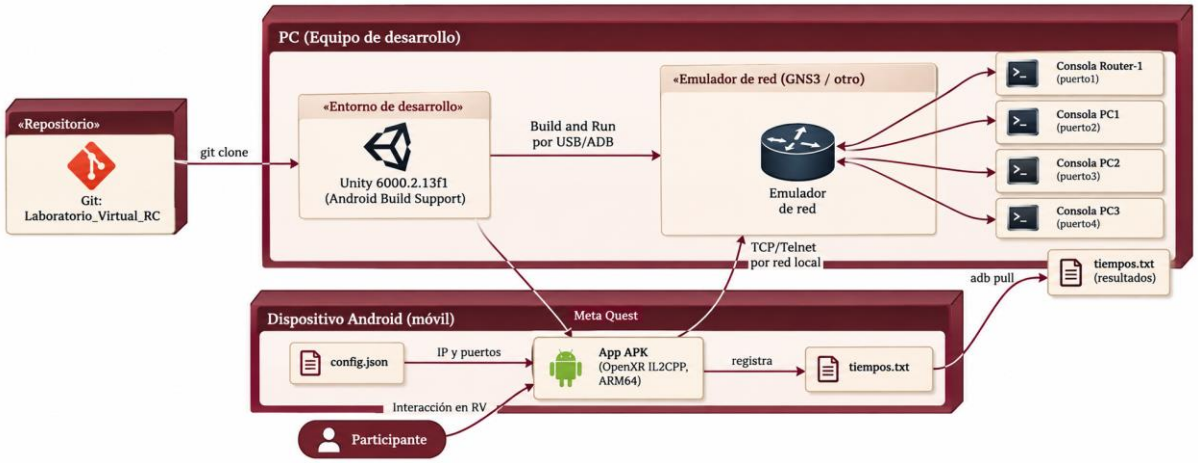
En primer lugar, USB y ADB permiten transferir e instalar la aplicación, además de copiar archivos de configuración y extraer los registros almacenados.

En segundo lugar, la comunicación TCP/Telnet permite que la aplicación ejecutada en Meta Quest establezca conexiones con las consolas de red disponibles en el computador.

El archivo config.json proporciona los parámetros de conexión, mientras que tiempos.txt almacena los registros de duración producidos por determinadas actividades del laboratorio.

De esta manera, el diagrama evidencia la separación entre el entorno de desarrollo, la infraestructura de simulación de red y el dispositivo de ejecución inmersiva.

Figura 1. Diagrama de despliegue del Laboratorio Virtual de Redes.



Fuente: Elaboración propia a partir de la estructura y configuración técnica del proyecto Laboratorio Virtual de Redes.

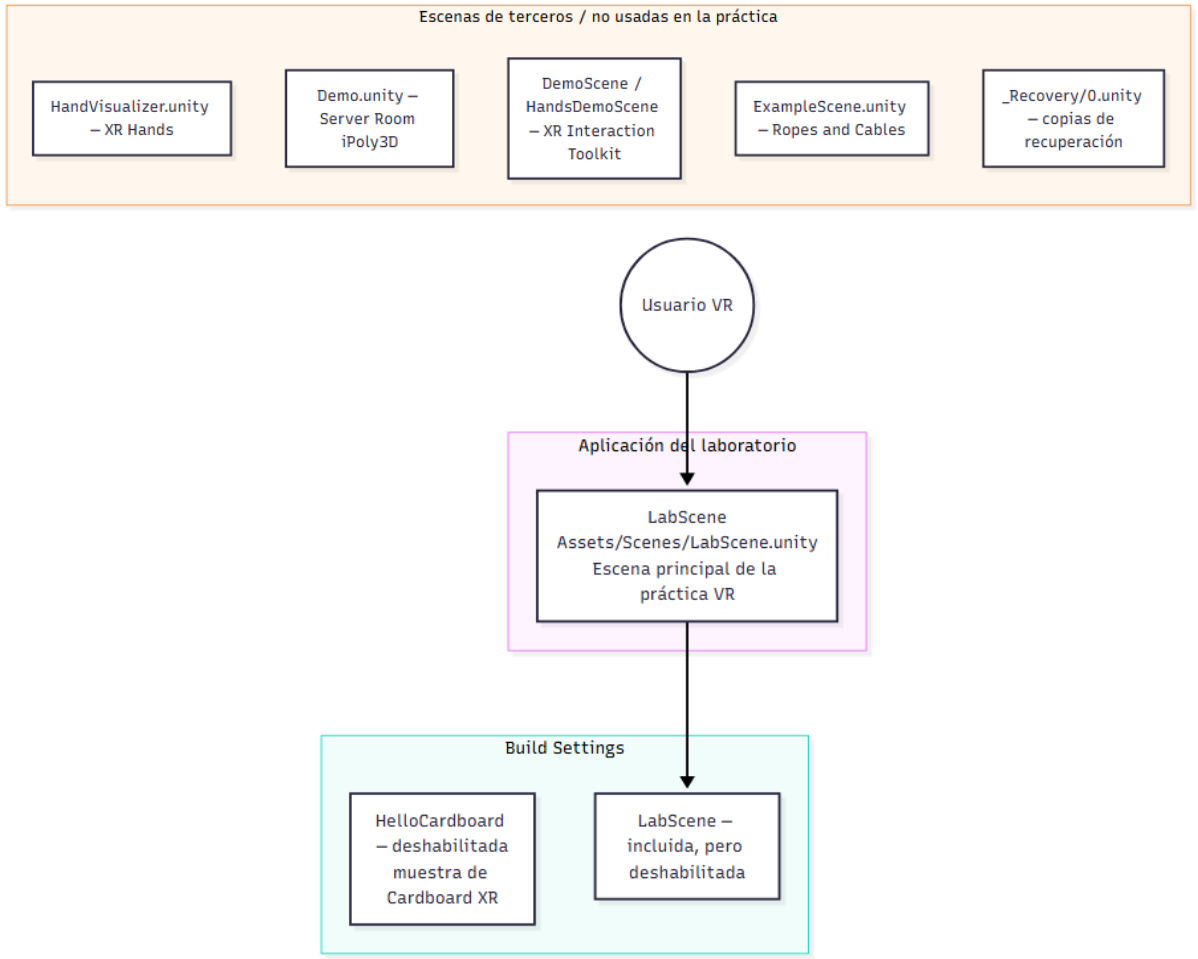
9. ARQUITECTURA DEL SOFTWARE

El laboratorio virtual se implementó en Unity como un entorno de realidad virtual. La arquitectura no se organiza mediante múltiples pantallas o niveles independientes, sino mediante una escena principal y un conjunto de componentes de software que controlan el avance de la práctica. Esta organización responde a un criterio pedagógico: reducir la carga cognitiva extrínseca asociada a cambios de interfaz y dosificar las tareas en fases sucesivas.

A continuación se presentan tres diagramas. El primero describe las escenas del proyecto. El segundo describe los componentes propios del software y sus relaciones. El tercero describe el flujo lógico de la práctica, es decir, el recorrido que realiza el usuario aunque permanezca en la misma escena.

9.1. Organización de las escenas

Figura 2. Organización de las escenas del proyecto Laboratorio Virtual de Redes.



Fuente: Elaboración propia a partir de la estructura y configuración técnica del proyecto Laboratorio Virtual de Redes.

El diagrama de escenas distingue tres conjuntos: la escena de la aplicación, las escenas registradas en la configuración de compilación (Build Settings) y las escenas auxiliares que no forman parte del producto.

9.1.1. Escena principal: LabScene

La práctica se ejecuta íntegramente en la escena LabScene, ubicada en Assets/Scenes/LabScene.unity. En ella coexisten el entorno tridimensional del laboratorio, los marcadores de inducción, los equipos del rack, el cableado y la interfaz del CLI.

Esta decisión de diseño implica que el usuario no cambia de escena al pasar de una etapa a otra. Lo que cambia es el estado interno del software: se ocultan o muestran marcadores, se habilitan paneles y se activan nuevas tareas. El espacio físico virtual se mantiene constante. Ello favorece la orientación espacial y evita interrupciones propias de una navegación por menús o por niveles.

9.1.2. Configuración de compilación

En EditorBuildSettings aparecen dos escenas:

Escena	Ruta	Estado
LabScene	Assets/Scenes/LabScene.unity	Registrada, deshabilitada
HelloCardboard	Muestra del Google Cardboard XR Plugin	Registrada, deshabilitada

LabScene es la escena que debe utilizarse para ejecutar y publicar el laboratorio. Su estado deshabilitado en la configuración de compilación no altera el diseño del sistema; es un ajuste de empaquetado. Para generar un ejecutable o una aplicación instalable es necesario habilitarla. HelloCardboard pertenece a una muestra del plugin de Cardboard y no interviene en el recorrido académico.

9.1.3. Escenas que no pertenecen a la práctica

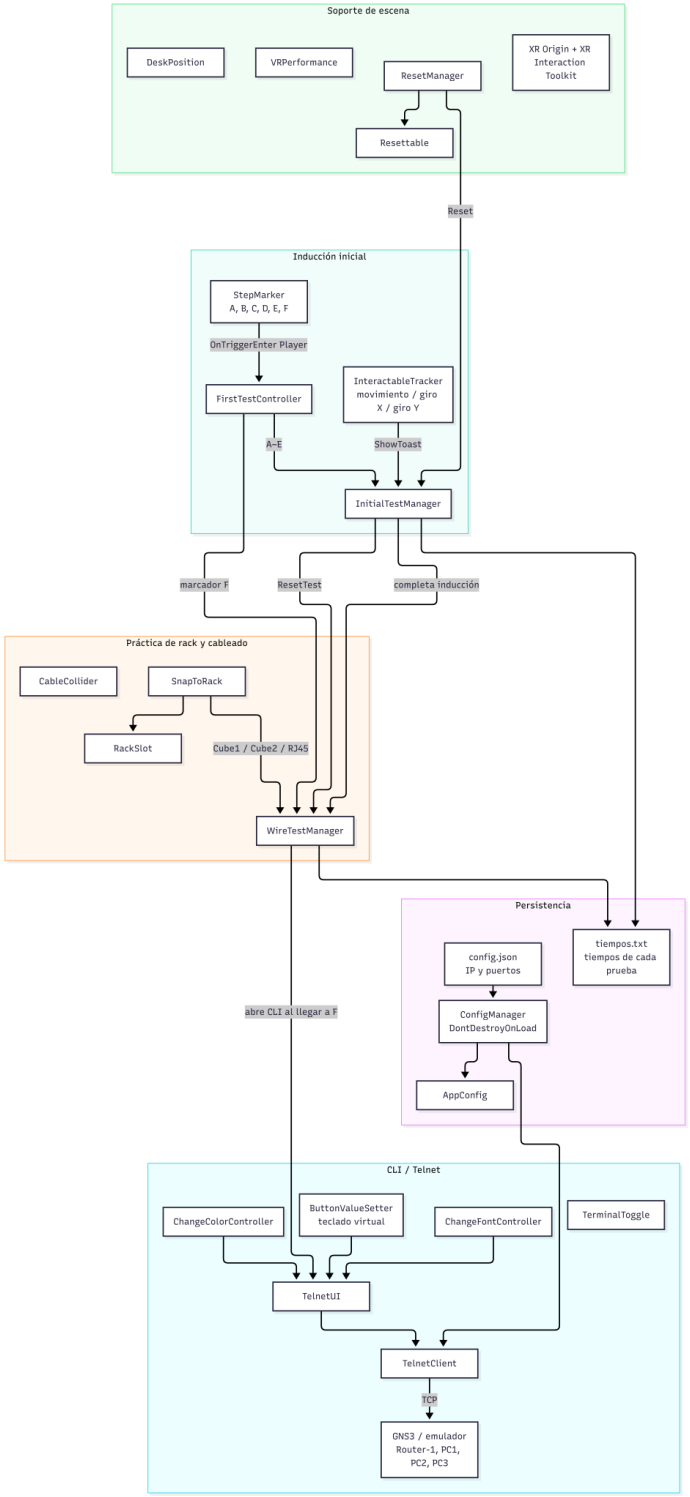
En el proyecto existen otras escenas asociadas a paquetes de terceros o a copias de recuperación:

- Demo.unity, del conjunto Server Room (iPoly3D), utilizada como referencia del entorno visual.
- DemoScene.unity y HandsDemoScene.unity, muestras de XR Interaction Toolkit.
- HandVisualizer.unity, muestra de XR Hands.
- ExampleScene.unity, ejemplo de Optimized Ropes and Cables.
- Escenas en Assets/_Recovery, copias de recuperación del editor.

Estas escenas no constituyen módulos del laboratorio. Se conservan porque Unity las incluye al importar paquetes o porque sirven de respaldo, pero el usuario de la práctica no las recorre.

9.2. Componentes del software

Figura 3. Diagrama de componentes del software.



Fuente: Elaboración propia a partir de la estructura y configuración técnica del proyecto Laboratorio Virtual de Redes.

Los scripts desarrollados para el proyecto se agrupan en cinco bloques: persistencia, inducción, montaje y cableado, CLI/Telnet y soporte de escena. El diagrama no incluye los scripts de los

paquetes de Unity (XR Interaction Toolkit, XR Hands, etc.), porque estos proveen infraestructura de interacción y no la lógica académica de la práctica.

9.2.1. Persistencia

Componente	Función
AppConfig	Estructura de datos con la dirección IP y los puertos de los equipos emulados (Router-1, PC1, PC2 y PC3).
ConfigManager	Carga config.json desde la ruta persistente de la aplicación y mantiene la configuración como instancia única (singleton). Permanece entre cargas de escena (DontDestroyOnLoad). Si el archivo no existe, asigna valores por defecto.
tiempos.txt	Archivo de registro. Almacena fecha, tipo de prueba y duración en segundos.

La separación entre configuración y código permite cambiar la IP o los puertos del emulador sin recompilar la aplicación. El registro de tiempos habilita un análisis posterior del desempeño, coherente con el interés por medir no solo el acierto, sino el esfuerzo temporal de cada bloque de actividad.

9.2.2. Inducción inicial

La inducción verifica que el usuario comprende el espacio y puede manipular objetos antes de iniciar el ejercicio de red.

Componente	Función
StepMarker	Collider asociado a un marcador (A, B, C, D, E o F). Cuando el objeto con etiqueta Player entra en el volumen, notifica al controlador y se desactiva para no repetir el evento.
FirstTestController	Recibe el nombre del marcador. Si el marcador es F, envía el evento a WireTestManager; en cualquier otro caso lo envía a InitialTestManager.

Componente	Función
InitialTestManager	Gestor de la inducción. Actualiza los indicadores de la interfaz, reproduce retroalimentación sonora, muestra mensajes (toasts), habilita los puntos D y E cuando corresponde, y guarda los tiempos de los grupos A-B-C y movimiento/rotación.
InteractableTracker	Observa un objeto agarrable. Al soltarlo, determina si hubo desplazamiento, giro en X o giro en Y por encima de un umbral, y reporta el evento al gestor de inducción. Cada tipo de acción se registra una sola vez hasta que se restablece la práctica.

El flujo de un marcador es lineal: collider, controlador, gestor, interfaz y archivo de tiempos. El flujo de manipulación es: agarre y suelte, umbrales, gestor. Ambas vías convergen en InitialTestManager, que actúa como núcleo de esta etapa.

Cuando se completan los marcadores A, B y C, el sistema habilita el punto D y comunica que puede continuarse con la prueba de acciones. Cuando se completan movimiento, giro en X y giro en Y, habilita el punto E e indica el paso a la prueba de cableado. Si ambas pruebas de inducción están completas, oculta el lienzo de inducción y muestra el de la etapa siguiente.

9.2.3. Montaje en rack y cableado

Componente	Función
RackSlot	Representa una ranura del rack. Indica si está ocupada y cambia de material para resaltar el espacio disponible.
SnapToRack	Se asigna a los objetos colocables. Al soltar el objeto cerca de una ranura libre, lo encaja, fija su transformación y notifica a WireTestManager con el nombre del objeto (Cube1, Cube2 o RJ45).
CableCollider	Genera colisiones por segmentos a lo largo de la malla del cable, de modo que este pueda detectarse e interactuarse en VR.
WireTestManager	Gestor del ejercicio de laboratorio. Valida la colocación de los dos elementos del rack y la conexión del cable; registra el tiempo de esta prueba; habilita el marcador F al completar las tres acciones; y, al llegar a F, activa el panel CLI, el panel de dispositivos, la ayuda y el teclado virtual.

WireTestManager es el segundo núcleo del sistema. No sustituye a InitialTestManager: lo sucede. El primero forma al usuario en el entorno; el segundo evalúa la tarea propia del laboratorio de redes.

9.2.4. CLI y conexión Telnet

Componente	Función
TelnetClient	Establece una conexión TCP con el emulador. Lee la IP desde ConfigManager y cambia de puerto según el dispositivo seleccionado (Router-1, PC1, PC2, PC3). Recibe datos en un hilo de lectura y los entrega a la interfaz en el hilo principal. Permite reconexión automática.
TelnetUI	Presenta la consola: limpia secuencias de control Telnet/ANSI, muestra la salida, envía comandos y gestiona el teclado virtual, incluidas mayúsculas y teclas especiales.
ButtonValueSetter	Asocia cada tecla del teclado virtual con el carácter o acción que debe enviarse a TelnetUI.
ChangeFontController	Cambia la tipografía del CLI (Liberation Sans, Camingo, Inconsolata, Courier Prime).
ChangeColorController	Cambia el color del texto del CLI (naranja, cian, blanco, verde).
TerminalToggle	Permite mostrar u ocultar el lienzo de la terminal, por tecla o por botón.

Esta capa conecta el laboratorio virtual con la red emulada. El estudiante no configura un dispositivo ficticio dibujado en Unity; envía comandos reales a través de Telnet hacia los puertos definidos en config.json. Los controles de fuente y color no alteran la lógica de red: apoyan la legibilidad y la accesibilidad, en línea con la reducción de carga extrínseca.

9.2.5. Soporte de la escena

Componente	Función
Resettable	Guarda la posición, rotación, escala y estado inicial de un objeto, y los restaura. Si el objeto tiene InteractableTracker, también reinicia sus banderas.
ResetManager	Restaura el conjunto de objetos, reactiva marcadores de inducción, oculta marcadores de avance y solicita a InitialTestManager un reset global. Ese reset, a su vez, llama a WireTestManager.ResetTest().

Componente	Función
DeskPosition	Sitúa el origen XR en una posición de escritorio y puede desactivar la locomoción.
VRPerformance	Ajusta la tasa de fotogramas objetivo (72 fps) y la escala de resolución de los ojos, para un rendimiento más estable en visor.
Player / XRSimulatorController	Apoyo de desplazamiento y simulación de controles cuando no se utiliza un visor, o para pruebas en escritorio.

El reset es un mecanismo de repetición. Permite que varios estudiantes, o el mismo estudiante en un segundo intento, partan de un estado conocido sin recargar LabScene.

9.2.6. Relación entre los dos gestores

El diagrama de componentes debe leerse alrededor de dos núcleos:

1. InitialTestManager: controla marcadores A–E, acciones de manipulación, mensajes de inducción y el reset general.
2. WireTestManager: controla Cube1, Cube2, RJ45, el marcador F y la apertura del CLI.

FirstTestController es el puente entre el espacio físico (marcadores) y esos dos gestores. ConfigManager es el puente entre la aplicación y el emulador de red. Los archivos config.json y tiempos.txt son la frontera con el sistema de archivos.

9.3. Flujo lógico de la práctica

Figura 4. Flujo lógico de la práctica.



Fuente: Elaboración propia a partir de la estructura y configuración técnica del proyecto Laboratorio Virtual de Redes.

Aunque Unity mantiene una sola escena, la experiencia se divide en cuatro fases. Cada fase se desbloquea únicamente cuando la anterior ha sido completada. El diagrama representa ese encadenamiento, no un cambio de archivo .unity.

9.3.1. Fase 1. Orientación espacial (marcadores A, B y C)

El usuario debe atravesar tres puntos del laboratorio. El sistema registra cada paso, muestra una confirmación breve y mide el tiempo desde el primer marcador hasta completar los tres. Al terminar, habilita el punto D y comunica que puede continuarse con la prueba de acciones.

Propósito: verificar que el estudiante se orienta en el entorno antes de pedirle tareas de mayor complejidad.

9.3.2. Fase 2. Manipulación de objetos (movimiento, giro en X y giro en Y)

En el punto D, el usuario debe desplazar un objeto y rotarlo en dos ejes, superando umbrales definidos en InteractableTracker. Cada acción se cuenta una vez. Al completar las tres, el sistema habilita el punto E.

Propósito: confirmar que el estudiante domina la interacción XR (agarrar, mover y soltar) antes de montar equipos de red.

9.3.3. Fase 3. Montaje y cableado (Cube1, Cube2 y RJ45)

En el punto E comienza el ejercicio propio del laboratorio. El usuario coloca dos elementos en el rack y conecta el cable de consola. SnapToRack valida la colocación. Al completar las tres acciones se habilita el punto F.

Propósito: trasladar la destreza espacial a una tarea reconocible de un laboratorio de redes: instalar y conectar.

9.3.4. Fase 4. Configuración por CLI (marcador F)

Al llegar a F se oculta el marcador, se activan el panel CLI, el panel de dispositivos, la ayuda y el teclado, y el usuario comienza a enviar comandos por Telnet hacia el emulador.

Propósito: realizar la actividad de mayor demanda cognitiva —la configuración de red— cuando el entorno, la manipulación y el montaje ya han sido resueltos.

9.3.5. Encadenamiento y fundamento pedagógico

El orden A-B-C → D → manipulación → E → rack/cable → F → CLI no es arbitrario. Se diseña para no presentar de forma simultánea orientación, destreza motriz, montaje físico y sintaxis de comandos. De acuerdo con la teoría de la carga cognitiva, esta dosificación reduce la carga extrínseca (esfuerzo que no aporta al contenido) y reserva recursos para la carga cognitiva pertinente (germane), es decir, la construcción del conocimiento de redes.

Los tiempos se almacenan por grupo:

- Marcadores A-B-C.
- Movimientos M, RX y RY.
- Ejercicio de cableado.

Este desglose permite identificar en qué fase aumenta la dificultad, en lugar de disponer únicamente de un tiempo global.

9.4. Síntesis

El software del laboratorio virtual puede resumirse en cuatro afirmaciones:

1. Una escena (LabScene) contiene toda la experiencia; las demás escenas del proyecto no pertenecen al recorrido del usuario.
2. Dos gestores coordinan el avance: InitialTestManager en la inducción y WireTestManager en el ejercicio de laboratorio.

3. Una conexión Telnet, configurada mediante config.json, vincula el entorno virtual con los equipos emulados.
4. Cuatro fases lógicas dosifican la práctica, de la orientación espacial hasta la configuración de red, y registran tiempos por bloque de actividad.

El laboratorio, por tanto, no se presenta como un conjunto de pantallas, sino como un mismo entorno que se revela progresivamente, de menor a mayor complejidad.

10. INTERACCIONES Y SCRIPTS

10.1. Propósito de la sección

Este apartado explica cómo se mueve e interactúa una persona dentro del laboratorio virtual, cómo se agarran y colocan los equipos, cómo se utilizan los paneles y teclados, y qué scripts coordinan cada parte. La intención es que cualquier persona que no haya escrito el código pueda comprender el recorrido de una interacción desde que el usuario realiza una acción hasta que la escena muestra el resultado.

La documentación se elaboró a partir de los scripts, los paquetes de Unity y las referencias visibles en la escena principal. Cuando una capacidad aparece en un script, pero no se confirmó que esté conectada en la escena, se indica expresamente. Esto es importante porque en Unity un componente puede existir en el proyecto y aun así no ejecutarse si no se agrega a un objeto o no recibe sus referencias desde el Inspector.

10.2. Resumen de la experiencia de interacción

El usuario completa ejercicios por etapas. Primero recorre el espacio y activa marcadores. Después mueve o gira objetos para que el sistema registre esas acciones. En la etapa de cableado toma equipos y cable, los coloca en posiciones del rack y, cuando los elementos requeridos están listos, continúa hacia la consola CLI.

La interacción se apoya en cuatro capas:

1. **Entrada del usuario:** controladores o manos XR, locomoción y rayos de interacción con la interfaz. También existe un script de simulación con teclado y ratón para PC.
2. **Detección en el mundo:** colliders, triggers, XRGrabInteractable y Rigidbody permiten detectar proximidad, agarre, liberación y contacto con botones o marcadores.
3. **Reglas del ejercicio:** managers y scripts de seguimiento convierten esas acciones en estados, como marcador visitado o equipo colocado.
4. **Respuesta visible:** paneles, toggles, mensajes temporales, resaltados, sonidos y cambios de etapa comunican el resultado.

10.3. Dependencias del proyecto

Las principales dependencias utilizadas son las siguientes:

- **Unity 6000.2.13f1:** motor utilizado por el proyecto.

- **XR Interaction Toolkit 3.2.2:** permite el agarre de objetos, la selección y la interacción con elementos de interfaz en realidad virtual.
- **Input System 1.14.2:** gestiona las acciones de entrada. El script Player.cs utiliza PlayerInput y las acciones Move y Jump.
- **OpenXR 1.16.0:** proveedor de realidad extendida indicado por los paquetes y ajustes XR.
- **XR Hands 1.7.3:** paquete disponible para funciones de manos XR. Su presencia no confirma por sí sola qué modalidad de manos está activa.
- **UGUI 2.0.0:** permite utilizar Canvas, botones, toggles, campos de entrada, desplegables y ScrollRect.
- **TextMesh Pro:** proporciona texto para la consola, etiquetas y controles de interfaz.
- **Física de Unity:** gestiona cuerpos rígidos, colliders, triggers, gravedad y velocidad de los objetos.
- **TCP/.NET:** permite la comunicación de la consola mediante TelnetClient.

10.4. Controles e interacción del usuario

10.4.1. Interacción en realidad virtual

La escena incluye un rig XR. Los objetos que pueden agarrarse deben tener XRGrabInteractable y un collider compatible. Para que el comportamiento de soltar y encajar funcione, también deben contar con Rigidbody y con el script SnapToRack cuando corresponda.

Las principales interacciones son:

- **Tomar un objeto:** el usuario apunta o acerca el interactor XR al objeto y activa el agarre configurado en el controlador o dispositivo. El toolkit genera el evento de selección.
- **Mover el objeto:** mientras permanece agarrado, el objeto sigue al interactor según las opciones configuradas en XRGrabInteractable.
- **Soltar el objeto:** al terminar la selección se ejecuta la lógica de liberación. Si el objeto está sobre una posición libre del rack, el script lo encaja; de lo contrario, queda sujeto a la física normal.
- **Pulsar botones físicos:** ButtonVR responde cuando un collider entra en el volumen del botón y cuando sale. Los eventos de pulsación y liberación permiten enlazar acciones desde el Inspector.
- **Utilizar paneles:** la escena contiene los Canvas DeviceCanvas, CLICanvas, WireCanvas y ResetCanvas. Algunos incluyen TrackedDeviceGraphicRaycaster, necesario para que los dispositivos rastreados puedan apuntar a elementos de interfaz. La interacción también requiere un EventSystem, un módulo de entrada apropiado y un interactor XR configurado.

- **Entrar en el área de escritura:** TypingArea muestra la mano virtual correspondiente cuando detecta objetos etiquetados como LeftHand o RightHand dentro del volumen. Al salir, vuelve a ocultarla.
- **Ir a la mesa:** DeskPosition mueve el XROrigin a una posición configurada cuando se activa su Toggle. Mientras el Toggle está activo, deshabilita el LocomotionProvider asignado; al desactivarlo, vuelve a habilitarlo.

Los nombres concretos de los botones físicos del controlador, como grip o trigger, dependen de las acciones y bindings configurados en el Inspector. Los scripts utilizan los eventos estándar de selección del XR Interaction Toolkit y no establecen un nombre universal para cada botón físico.

10.4.2. Simulación en PC con teclado y ratón

El archivo Assets/Scripts/KeyboardVRCameraController.cs define la clase XRSimulatorController. Este script permite mover un rig y sus controladores simulados mediante teclado y ratón, siempre que se agregue a un GameObject y se asignen las referencias xrRig, mainCamera, leftController y rightController en el Inspector.

Los controles definidos en el script son:

- Mover el rig horizontalmente: W, A, S y D.
- Girar el rig a izquierda o derecha: Q y E.
- Mirar alrededor: movimiento del ratón.
- Mostrar u ocultar el cursor: tecla ESC.
- Desplazar el controlador izquierdo en el eje X: teclas 4 y 6 del teclado numérico.
- Desplazar el controlador izquierdo en el eje Z: teclas 8 y 5 del teclado numérico.
- Desplazar el controlador izquierdo en el eje Y: teclas 7 y 9 del teclado numérico.
- Simular grip izquierdo: mantener pulsada la tecla G.
- Simular trigger izquierdo: mantener pulsado el botón derecho del ratón.
- Desplazar el controlador derecho en el eje X: teclas J y L.
- Desplazar el controlador derecho en el eje Z: teclas I y K.
- Desplazar el controlador derecho en el eje Y: teclas U y O.
- Simular grip derecho: mantener pulsada la tecla H.
- Simular trigger derecho: mantener pulsado el botón izquierdo del ratón.

Es importante señalar que no se encontró un componente XRSimulatorController referenciado en LabScene.unity. Por tanto, los controles anteriores describen las funciones escritas en el script, pero no confirman que estén disponibles al ejecutar el proyecto tal como está.

Además, los métodos de simulación de grip y trigger comprueban si existe un XRDirectInteractor y escriben un mensaje en la consola de depuración, pero no activan por sí mismos el mecanismo de selección XR. Para que funcionen como una simulación efectiva del agarre, deben conectarse con la entrada y la interacción configuradas en el rig.

El simulador utiliza UnityEngine.Input y los ejes clásicos Horizontal, Vertical, Mouse X y Mouse Y. Como el proyecto también incluye Input System, es necesario revisar la compatibilidad si Active Input Handling está configurado exclusivamente para el nuevo sistema.

10.4.3. Otros controles de PC

Player.cs es un controlador distinto que obtiene PlayerInput, lee la acción Move en cada actualización y aplica fuerza al Rigidbody. Su método Jump aplica un impulso cuando la acción se completa. Para funcionar requiere que el mismo GameObject tenga PlayerInput y Rigidbody, además de un mapa de acciones que incluya Move y Jump. No se confirmó que sea el controlador activo del rig XR principal.

Por otra parte, TerminalToggle.cs permite mostrar u ocultar el Canvas del terminal mediante la tecla T, siempre que el componente y la referencia al Canvas estén asignados. En la ruta CLI también se puede utilizar un campo de texto con teclado físico, además del teclado virtual.

10.5. Agarre, liberación y colocación de objetos en el rack

10.5.1. Componentes involucrados

Los principales componentes que participan en la colocación de equipos son:

- **XRGrabInteractable:** permite que el objeto participe en la selección y el agarre del XR Interaction Toolkit.
- **Collider:** define la forma que detectan el interactor y los slots.
- **Rigidbody:** aporta gravedad y física cuando el objeto está libre. Al encajarlo, el script lo vuelve cinemático.
- **SnapToRack:** escucha los eventos de agarre y liberación, guarda los slots cercanos y realiza el encaje.
- **RackSlot:** representa una posición del rack, conserva su estado de ocupación y cambia su material para indicar disponibilidad o selección.

Estos componentes deben estar en los objetos correctos y sus colliders deben configurarse como triggers o cuerpos físicos según el montaje de la escena. Los slots necesitan un collider para que puedan detectarse y para que el script calcule su centro.

10.5.2. Proceso de agarre

Cuando el interactor selecciona un objeto, SnapToRack.OnGrab marca que el objeto está siendo agarrado. Si el objeto ya estaba colocado en un slot, la posición anterior vuelve a marcarse como libre y se elimina su resaltado.

Después, el objeto se desvincula del slot (deja de ser su objeto hijo) y se reactiva la física: isKinematic queda en falso y la gravedad se activa. Esto permite que el objeto vuelva a seguir al controlador y que, al soltarlo fuera del rack, caiga o se mueva como un objeto normal.

10.5.3. Proceso de colocación

Mientras el collider del objeto entra en un slot desocupado, SnapToRack incorpora ese slot a su lista de candidatos y lo resalta. Si el objeto sale del volumen antes de soltarlo, el slot deja de estar en la lista y se elimina su resaltado.

Cuando termina el agarre, SnapToRack.OnRelease busca el slot candidato libre más próximo comparando la distancia entre el objeto y el centro del collider del slot.

Si encuentra un slot disponible, realiza las siguientes acciones:

1. Convierte el slot en el objeto padre del equipo.
2. Coloca el objeto en el centro del collider del slot o en el transform del slot si no tiene collider.
3. Restaura la rotación inicial capturada al iniciar el objeto.
4. Marca el slot como ocupado y conserva el resaltado.
5. Desactiva la gravedad y convierte el Rigidbody en cinemático.
6. Detiene las velocidades lineal y angular para evitar vibraciones o desplazamientos posteriores.
7. Envía el nombre del objeto a WireTestManager.Instance.ShowToast(...).

Si no existe un slot candidato, el objeto conserva la física libre. No se observó una lógica que lo devuelva automáticamente a su posición inicial al soltarlo. Ese comportamiento corresponde a Resettable cuando se acciona el reinicio.

10.5.4. Registro del progreso del ejercicio

WireTestManager reconoce los nombres exactos Cube1, Cube2 y RJ45. Cuando recibe estos nombres, activa los toggles asignados a los elementos del ejercicio, muestra un mensaje y comprueba si los tres están completos.

Cuando se completa el conjunto, activa el marcador F y muestra un mensaje que invita a continuar en la consola. El tiempo transcurrido se guarda en el archivo tiempos.txt dentro de Application.persistentDataPath.

Por este motivo, el nombre de cada GameObject que se puede agarrar forma parte de la lógica del ejercicio. Si se cambia un nombre en la escena, el encaje puede seguir funcionando, pero el manager no reconocerá el objeto a menos que se actualice la condición correspondiente.

10.6. Interacción con botones y elementos de interfaz

10.6.1. Interfaz gráfica en Canvas

UGUI organiza los controles en Canvas. Para que sean interactivos en realidad virtual no basta con que sean visibles: se necesita un sistema de eventos que reciba la entrada del controlador y que el Canvas pueda ser alcanzado.

TrackedDeviceGraphicRaycaster proporciona el raycast gráfico para dispositivos XR. El rig debe tener su interactor de interfaz y la escena debe disponer de un EventSystem con un módulo de entrada compatible.

Los Canvas identificados en la escena son:

- **DeviceCanvas:** panel de selección e interacción con dispositivos.
- **CLICanvas:** panel de consola e ingreso de comandos.
- **WireCanvas:** interfaz asociada al ejercicio de cableado.
- **ResetCanvas:** controles de reinicio.
- **Otros Canvas:** paneles de descripción, mensajes y ajustes que ayudan a orientar al usuario.

Los scripts de gestión reciben y actualizan los objetos de interfaz mediante referencias serializadas. En el Inspector deben asignarse el panel de mensajes temporales, los textos, toggles, marcadores y Canvas objetivo. Una referencia vacía puede impedir que una acción actualice la interfaz o provocar un error.

10.6.2. Botón físico de realidad virtual

ButtonVR utiliza un objeto visual llamado button, dos eventos denominados onPress y onRelease, y un AudioSource.

Cuando un collider entra en el trigger del botón, el script desplaza el objeto visual para simular que se hunde, guarda qué collider lo pulsó, invoca onPress, reproduce el sonido y evita repetir la pulsación mientras el botón continúa presionado.

Cuando ese mismo collider sale, restaura la posición del objeto visual, invoca onRelease y permite una nueva pulsación.

Los eventos de Unity pueden enlazarse a funciones desde el Inspector. Por ejemplo, los botones del teclado virtual utilizan onRelease para insertar una tecla. ButtonVR.SpawnSphere() existe como función de ejemplo, pero no forma parte del flujo normal del teclado.

10.6.3. Teclado virtual para la consola CLI

La interacción del teclado virtual se organiza mediante varios scripts:

- **KeyboardCLI:** agrega caracteres, espacios o elimina el último carácter del TMP_InputField. También alterna los grupos visuales de teclas normales y mayúsculas.
- **KeyboardButton:** toma el texto de la tecla y, si corresponde a un carácter individual, enlaza la liberación del botón VR con KeyboardCLI.InsertChar.
- **ButtonVR:** detecta el contacto físico y produce los eventos del botón.
- **TypingArea:** muestra las manos virtuales de escritura cuando las manos etiquetadas entran en el área del teclado.

Existe también otra ruta de teclado vinculada a la consola Telnet. ButtonValueSetter identifica la tecla por el nombre del GameObject y llama a TelnetUI.VirtualKeyPress. En esta ruta, las teclas Caps, Enter, Retroceso, Espacio y las flechas tienen un tratamiento especial. El resto de las teclas se añade al campo de entrada.

10.7. Flujo de etapas, mensajes y reinicio

10.7.1. Marcadores y prueba inicial

StepMarker detecta el evento OnTriggerEnter y solo responde si el objeto que entra tiene la etiqueta Player. El nombre del marcador se envía a FirstTestController, que distribuye el evento de la siguiente manera:

- Los marcadores distintos de F se envían a InitialTestManager.
- El marcador F se envía a WireTestManager.

InitialTestManager gestiona los marcadores A, B y C, y posteriormente las acciones de movimiento y giro. Para los marcadores A, B y C activa los toggles de progreso y controla el acceso a los puntos siguientes.

Para la prueba de manipulación, InteractableTracker registra la posición y la rotación iniciales al comenzar el agarre. Al soltar el objeto, compara el desplazamiento y los giros con los umbrales configurados moveThreshold y rotateThreshold.

Cada logro se notifica una sola vez hasta que se reinician los indicadores de progreso. Al completar las acciones, el manager habilita el marcador E para avanzar a la etapa de cableado.

Los mensajes se presentan en un panel temporal con animaciones de aparición y desaparición, sonidos de éxito o finalización y cambios de Canvas después de una demora. Los tiempos de las pruebas se guardan en tiempos.txt junto con la fecha, el grupo y la duración.

10.7.2. Etapa de cableado y consola CLI

WireTestManager mantiene los toggles del rack para Cube1, Cube2 y RJ45. Cuando los tres objetos están colocados, activa el marcador F. Al llegar a este marcador, muestra los paneles de ayuda, CLI, dispositivos y teclado.

La consola combina los siguientes scripts:

- **TelnetUI:** gestiona la entrada de comandos, el teclado virtual, la limpieza del texto recibido y la actualización del desplazamiento de la consola.
- **TelnetClient:** administra la conexión TCP con el host y el puerto, la lectura en un hilo de fondo, los reintentos de conexión y la entrega de eventos a la escena.
- **ConfigManager:** carga config.json desde Application.persistentDataPath. Si el archivo no existe, establece valores locales predeterminados.
- **ButtonValueSetter:** conecta los botones gráficos del teclado con la entrada de TelnetUI.

TelnetClient agrega CRLF al texto enviado. TelnetUI elimina las secuencias de control ANSI y los caracteres no imprimibles del texto recibido antes de presentarlo.

La selección del puerto se basa en los nombres de opciones como Router-1, PC1, PC2 y PC3, y utiliza los números de puerto de la configuración cargada.

10.7.3. Reinicio del laboratorio

ResetManager.ResetAll() recorre los objetos Resettable asignados y restaura cada uno. También activa los marcadores que deben mostrarse, desactiva los que deben ocultarse y presenta el mensaje “Reset”.

Resettable guarda al inicio la posición, rotación, escala y estado activo del GameObject. Al reiniciar, restaura esos valores, pone las velocidades del Rigidbody en cero y limpia los indicadores de InteractableTracker.

El botón de reinicio es opcional y se conecta mediante la referencia resetButton en el Inspector. La lógica también puede invocarse desde Unity Events.

El reinicio de InitialTestManager desmarca los toggles de la primera prueba y reactiva su Canvas. Además, llama a WireTestManager.ResetTest() para cerrar los paneles y limpiar el estado del ejercicio de cableado.

10.8. Scripts principales y sus funciones

Los principales scripts identificados y sus responsabilidades son los siguientes:

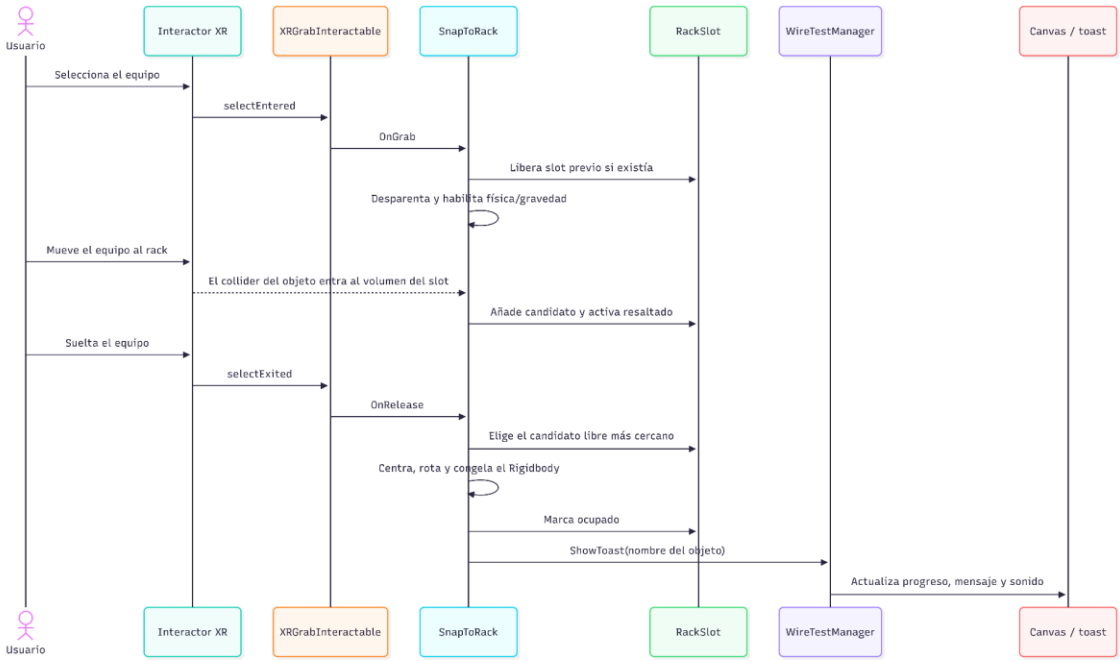
- **SnapToRack.cs:** recibe los eventos de selección, liberación y entrada o salida de triggers. Despega, encaja y mantiene quieto un objeto, además de informar su nombre al manager. Requiere XRGrabInteractable, Rigidbody, colliders, slots y WireTestManager.Instance.
- **RackSlot.cs:** registra la ocupación de los slots y cambia los materiales para resaltarlos. Requiere collider, Renderer y materiales correctamente asignados.
- **InteractableTracker.cs:** detecta el movimiento y el giro de los objetos al comenzar y finalizar una selección XR. Requiere XRGrabInteractable, umbrales de movimiento y rotación, e InitialTestManager.Instance.
- **Resettable.cs:** restaura la posición, rotación, escala, estado activo y velocidad de un objeto. Puede utilizar InteractableTracker y Rigidbody.
- **ResetManager.cs:** reinicia los objetos y marcadores configurados. Requiere las listas de objetos y marcadores, además del manager inicial.
- **KeyboardVRCameraController.cs (XRSimulatorController):** mueve el rig, la cámara y los controladores simulados mediante teclado y ratón. Debe agregarse a la escena y configurarse en el Inspector. No se encontró como componente referenciado en la escena principal.
- **Player.cs:** recibe las acciones Move y Jump del Input System y aplica fuerzas para el movimiento y el salto. Requiere PlayerInput, Rigidbody y un mapa de acciones correcto. No se confirmó su uso en el rig principal.
- **DeskPosition.cs:** teletransporta el XROrigin a la mesa y alterna la locomoción cuando cambia el Toggle. Requiere las referencias correspondientes.
- **StepMarker.cs:** detecta la entrada de un collider con la etiqueta Player, envía el nombre del marcador y se desactiva. Requiere la etiqueta correcta y FirstTestController en el objeto padre.

- **FirstTestController.cs:** recibe el nombre del marcador y lo dirige al manager de la etapa inicial o de cableado.
- **InitialTestManager.cs:** controla la primera prueba, los toggles, sonidos, mensajes temporales, marcadores y registro del tiempo. Requiere que los paneles, textos, toggles, marcadores y sonidos estén asignados en el Inspector.
- **WireTestManager.cs:** controla el rack, la etapa CLI, los paneles y el tiempo. Los nombres de los objetos deben coincidir exactamente con los esperados por el script.
- **ButtonVR.cs:** detecta la entrada y salida de colliders en un botón físico, anima la pulsación y dispara eventos de Unity. Requiere trigger, objeto visual, AudioSource y eventos configurados.
- **KeyboardCLI.cs:** inserta y elimina caracteres y cambia el grupo de teclas mostrado. Requiere un InputField y los objetos de teclas normales y mayúsculas.
- **KeyboardButton.cs:** conecta el carácter visual de una tecla con el teclado virtual. Requiere KeyboardCLI en el padre, texto y ButtonVR en el botón.
- **TypingArea.cs:** muestra u oculta las manos virtuales de escritura al detectar su entrada o salida de un trigger. Requiere las etiquetas LeftHand y RightHand y las referencias a los elementos visuales.
- **TelnetUI.cs:** envía comandos y presenta la salida limpia de la consola. Requiere TelnetClient, TMP_InputField, TMP_Text y ScrollRect.
- **TelnetClient.cs:** abre la conexión, recibe y envía texto y realiza reintentos. Requiere host, puerto, Dropdown, ConfigManager y un servidor accesible.
- **ButtonValueSetter.cs:** envía el valor de una tecla de la interfaz a TelnetUI. Utiliza el nombre del objeto para identificar la tecla y requiere Button y un texto hijo.
- **ConfigManager.cs y AppConfig.cs:** cargan la configuración de IP y puertos desde un archivo JSON o utilizan valores predeterminados. El archivo config.json es opcional y se ubica en persistentDataPath.
- **TerminalToggle.cs:** muestra u oculta el Canvas del terminal mediante la tecla T o llamadas públicas. Requiere un Canvas asignado y utiliza el sistema clásico de entrada de Unity.
- **CableCollider.cs:** crea cápsulas de colisión a lo largo de la malla del cable. Requiere MeshFilter y parámetros de segmentos y radio.
- **ChangeColorController.cs:** cambia el color de los textos de la consola y del campo de entrada mediante un TMP_Dropdown.
- **ChangeFontController.cs:** cambia las fuentes disponibles mediante un TMP_Dropdown y requiere las referencias a los textos y cuatro TMP_FontAsset.

10.9. Diagramas de secuencia de las interacciones

10.9.1. Agarrar y colocar un equipo en el rack

Figura 5. Diagrama de secuencia: agarrar y colocar un equipo en el rack.



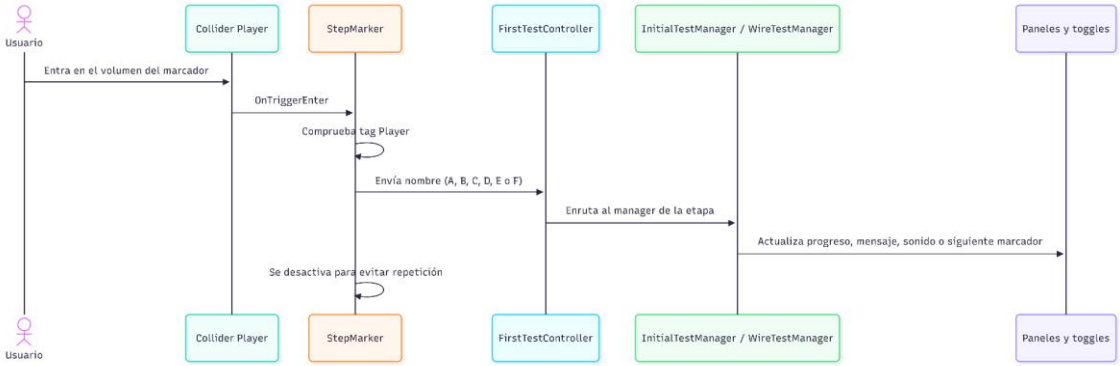
Fuente: Elaboración propia a partir del análisis de los scripts del proyecto Laboratorio Virtual de Redes.

El diagrama muestra el recorrido de un equipo desde que el usuario lo toma hasta que el laboratorio confirma que quedó colocado. El interactor XR (por ejemplo, una mano o un controlador) selecciona el objeto y el componente XRGrabInteractable comunica ese evento a SnapToRack. Al comenzar el agarre, el script libera el slot anterior si el objeto ya estaba guardado y permite que vuelva a moverse con física.

Mientras el usuario acerca el equipo al rack, los colliders detectan los slots disponibles. SnapToRack registra los candidatos cercanos y RackSlot proporciona la señal visual de resaltado. Al soltar, el script escoge el slot libre más próximo, centra y alinea el equipo, detiene su movimiento y marca el slot como ocupado. Finalmente, envía el nombre del objeto a WireTestManager, que actualiza el progreso y presenta un mensaje y un sonido. Si se suelta fuera de un slot válido, el equipo queda libre y continúa bajo la acción de la gravedad.

10.9.2. Visitar un marcador de etapa

Figura 6. Diagrama de secuencia: visitar un marcador de etapa.



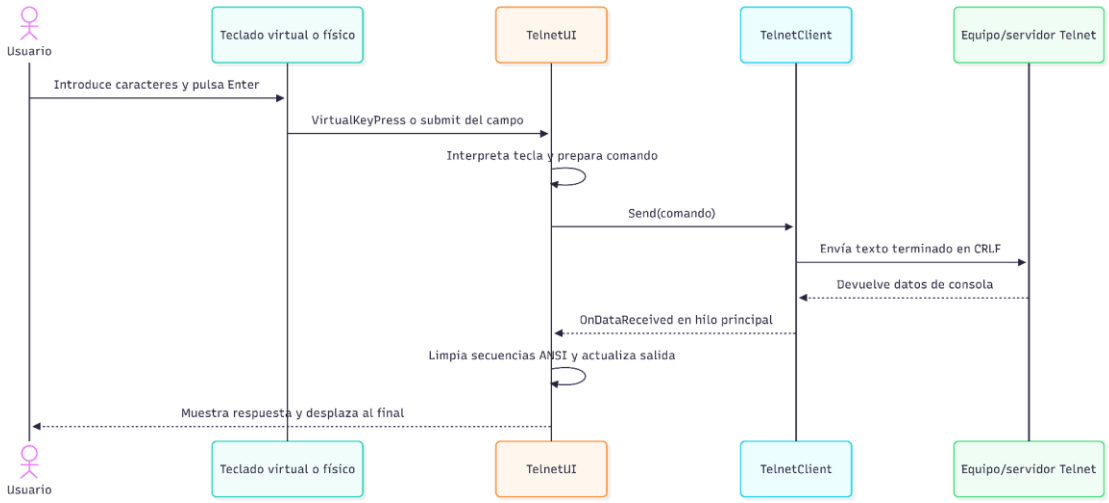
Fuente: Elaboración propia a partir del análisis de los scripts del proyecto Laboratorio Virtual de Redes.

Este diagrama describe cómo el sistema reconoce que el usuario llegó a un punto de la actividad. Cuando el collider del jugador entra en el volumen del marcador, StepMarker comprueba que el objeto tenga la etiqueta Player. Si la etiqueta coincide, toma el nombre del marcador y se lo envía a FirstTestController.

FirstTestController decide qué manager debe atender el evento: los marcadores de las pruebas iniciales se envían a InitialTestManager, mientras que el marcador F se dirige a WireTestManager. El manager correspondiente activa el progreso, muestra la instrucción o habilita el siguiente paso. El marcador se desactiva después de activarse para evitar que el mismo contacto cuente repetidamente.

10.9.3. Escribir y enviar un comando a la consola

Figura 7. Diagrama de secuencia: escribir y enviar un comando a la consola.



Fuente: Elaboración propia a partir del análisis de los scripts del proyecto Laboratorio Virtual de Redes.

El diagrama explica el camino de un comando desde el teclado hasta el equipo conectado y el regreso de su respuesta. El usuario escribe mediante el teclado virtual o el campo de texto. TelnetUI reúne las teclas y, cuando recibe Enter, prepara el comando. Luego lo entrega a TelnetClient, que agrega el terminador de línea CRLF y lo envía por la conexión TCP al equipo o servidor seleccionado.

Cuando llega una respuesta, TelnetClient la entrega al hilo principal de Unity y la comunica a TelnetUI. La interfaz limpia secuencias de formato ANSI y caracteres de control que no se necesitan, actualiza el texto visible y desplaza la consola hacia el final para mostrar la respuesta más reciente. Para que el recorrido funcione, la conexión debe estar disponible y las referencias del cliente, el campo de entrada, el texto y el scroll deben estar asignadas.

10.10. Configuración necesaria en Unity Inspector

Para que los flujos de interacción funcionen correctamente durante la ejecución, se deben revisar los siguientes aspectos:

1. **Objetos del rack:** cada equipo debe tener XRGrabInteractable, Rigidbody, collider y SnapToRack. Cada posición debe disponer de RackSlot y collider. También deben verificarse las capas y máscaras de interacción.
2. **Slots:** cada posición debe tener un Renderer accesible desde RackSlot. Se deben asignar los materiales normales y de resaltado, cuando corresponda, y comprobar que el estado inicial de ocupación sea correcto.
3. **Managers:** se deben asignar los marcadores, paneles, toggles, textos, sonidos y Canvas utilizados por InitialTestManager y WireTestManager. También debe confirmarse que sus instancias existan antes de que otros scripts llamen a Instance.
4. **Marcadores:** se debe comprobar que tengan un collider con Is Trigger activado, que el objeto detectado tenga la etiqueta Player y que FirstTestController esté en el objeto padre.
5. **Interacción de interfaz en VR:** se debe confirmar la existencia de EventSystem, módulo UI XR, interactor UI del controlador y TrackedDeviceGraphicRaycaster en cada Canvas interactivo.
6. **Botones físicos:** se deben validar el collider o trigger, el objeto visual que se desplaza, el AudioSource y los eventos onPress y onRelease.
7. **Consola:** se deben enlazar TelnetUI, TelnetClient, el campo de entrada, el texto de salida y ScrollRect. También se debe asignar el desplegable de puertos y verificar la IP, los puertos y la disponibilidad del servidor Telnet.
8. **Simulador de PC:** para utilizar XRSimulatorController, se debe agregar el componente y asignar sus cuatro referencias. También debe comprobarse la compatibilidad entre la entrada clásica y la configuración del Input System.
9. **Reinicio:** se deben completar las listas de objetos y marcadores de ResetManager, comprobar que los objetos tengan Resettable y verificar que su estado inicial se guarde después de colocarlos correctamente.

10.11. Observaciones y limitaciones identificadas

Durante la revisión del código se identificaron las siguientes observaciones:

- La clase XRSimulatorController está definida en un archivo llamado KeyboardVRCameraController.cs. Su existencia no demuestra que el componente esté activo, ya que no se encontró referenciado como componente en la escena principal.
- Los métodos SimularGrip y SimularTrigger registran la entrada en la consola, pero no fuerzan la selección del objeto. Por ello, el agarre mediante PC no debe considerarse equivalente al agarre real hasta que esas entradas se conecten al sistema de interacción.
- SnapToRack espera que los colliders que reciben los triggers puedan obtener RackSlot desde el mismo GameObject del collider. Si el collider está en un objeto hijo distinto del componente, habría que ajustar la búsqueda o la jerarquía.
- SnapToRack llama a WireTestManager.Instance.ShowToast al encajar cualquier objeto que tenga ese script. Sin embargo, el manager solo reconoce exactamente Cube1, Cube2 y RJ45. Un nombre diferente puede generar un aviso de mensaje desconocido.
- El script InteractableTracker asume que existe un XRGrabInteractable en el mismo objeto. El umbral se mide desde que comienza cada agarre hasta que se suelta, y los indicadores evitan que el mismo logro se contabilice nuevamente antes del reinicio.
- Resettable guarda el estado inicial en Awake. Por tanto, la posición almacenada debe ser la que se desea recuperar al reiniciar cuando se inicializan los objetos.
- ButtonVR responde ante cualquier collider que entre mientras el botón no esté presionado. Si se necesita aceptar únicamente una mano o herramienta específica, se debe aplicar un filtro por etiqueta, capa o componente.
- Los paquetes XR Hands, OpenXR y XR Interaction Toolkit están presentes en el proyecto, pero las capacidades efectivas también dependen del dispositivo objetivo, los loaders, los bindings y los componentes configurados en la escena.

10.12. Vocabulario técnico

- **Interactor:** controlador, mano o rayo que intenta seleccionar un objeto o elemento de interfaz.
- **Interactable:** objeto preparado para recibir interacciones, como un objeto con XRGrabInteractable.
- **Collider:** volumen invisible utilizado para detectar contacto o proximidad.
- **Trigger:** collider configurado para informar entradas y salidas sin bloquear físicamente al objeto.
- **Rigidbody cinemático:** cuerpo que no responde a fuerzas ni a la gravedad; se utiliza para mantener quieto un objeto una vez encajado.
- **Canvas:** superficie que contiene controles gráficos de la interfaz de Unity.
- **Inspector:** panel del editor de Unity donde se asignan referencias, valores, eventos y componentes.
- **Singleton (Instance):** mecanismo que permite acceder a una instancia compartida de un manager desde otros scripts.

11. CONFIGURACIÓN Y FUNCIONAMIENTO DEL TERMINAL TELNET

Este apartado explica cómo se integra la conexión Telnet con la escena de Unity: qué componente carga la configuración, cómo se selecciona un equipo, cómo viajan los comandos y las respuestas, y dónde quedan los datos guardados. La aplicación funciona como cliente: inicia una conexión TCP hacia un equipo remoto que debe tener un servicio Telnet escuchando en la IP y puerto seleccionados.

11.1. Componentes que participan

La funcionalidad está distribuida en varios scripts y objetos de interfaz. No hay un único módulo que haga todo; los componentes se comunican mediante referencias asignadas en el Inspector y eventos de Unity.

- **ConfigManager y AppConfig:** mantienen la configuración global de red durante la ejecución. ConfigManager es un singleton persistente entre escenas y carga el JSON al iniciar.
- **TelnetClient:** abre y mantiene el socket TCP, transmite texto, recibe bytes del equipo remoto y publica eventos de conexión y datos.
- **TelnetUI:** conecta los eventos del cliente con el panel visual. Prepara el texto recibido, lo presenta y reenvía los comandos escritos.
- **Campo de entrada y controles del teclado virtual:** capturan texto y lo entregan a TelnetUI para su envío.
- **TerminalToggle:** controla si el Canvas del terminal está visible. Ocultar el Canvas solo afecta a la presentación; no es el mecanismo que cierra el socket.
- **InitialTestManager y WireTestManager:** guardan localmente resultados de tiempo de ejercicios en un archivo de texto.

11.2. Cómo se configura la IP y los puertos

11.2.1. Carga de la configuración

Al iniciar, ConfigManager busca config.json en Application.persistentDataPath. Si lo encuentra, lee el texto JSON y lo convierte al objeto AppConfig, que contiene ip y cuatro enteros de puerto (puerto1 a puerto4). Al ser un singleton, otras escenas pueden consultar ConfigManager.Instance.Config sin volver a cargar el archivo.

```
{
  "ip": "192.168.1.50",
  "puerto1": 8080,
  "puerto2": 8081,
  "puerto3": 8082,
  "puerto4": 8083
}
```

El ejemplo ilustra la forma de los campos esperados; los números son valores de muestra, no valores universales. Si el archivo no existe, el código usa 127.0.0.1, puerto1 8080, puerto2

8081 y puerto3 8082. El parámetro puerto4 no recibe un valor en el bloque de valores predeterminados, por lo que queda en cero hasta que una configuración válida lo asigne.

11.2.2. Aplicación del host y selección del puerto

TelnetClient tiene valores iniciales declarados en el componente (host 127.0.0.1 y port 5000). En Start reemplaza el host con ConfigManager.Instance.Config.ip, registra el evento de cambio del desplegable y empieza el ciclo de conexión. El puerto se cambia cuando se procesa una selección del desplegable: Router-1 se asocia a puerto1, PC1 a puerto2, PC2 a puerto3 y PC3 a puerto4.

Esto implica que el puerto inicial 5000 del Inspector y los puertos predeterminados 8080–8082 son fuentes diferentes. El puerto que se use depende del valor del componente y de si se produce una selección del dropdown que llame a OnPortChanged. Al integrar o desplegar el proyecto, se deben revisar en la escena el valor inicial, las opciones exactas del dropdown y la configuración JSON para confirmar el destino efectivo.

11.2.3. Requisitos de red

La IP identifica el equipo destino desde el punto de vista de la máquina que ejecuta la aplicación. El puerto identifica el servicio Telnet dentro de ese equipo. El servicio remoto debe estar activo, aceptar conexiones TCP en ese puerto y ser alcanzable por la red local o ruta configurada. Un puerto que solo esté abierto en localhost no será accesible desde otro dispositivo; en ese caso debe configurarse la dirección de red del equipo servidor.

11.3. Arranque, conexión y reconexión

Cuando TelnetClient.Start se ejecuta, escucha cambios del puerto, toma la IP configurada e invoca Connect. Connect crea un hilo de fondo que ejecuta ConnectionLoop; así, la espera de red y la lectura no bloquean el hilo de renderizado de Unity. El intento de conexión tiene un límite de cinco segundos. Si lo supera, coloca un mensaje de fallo en la cola para la interfaz.

Cuando el socket conecta, TelnetClient obtiene el NetworkStream, encola el mensaje de conectado y programa el evento OnConnected. Después revisa periódicamente si hay datos disponibles, los lee en bloques de hasta 4096 bytes y los convierte desde ASCII. Cada bloque se encola para que Update lo entregue a OnDataReceived en el hilo principal, donde Unity puede actualizar con seguridad la interfaz.

Si el equipo cierra la conexión, ocurre un error o se cambia de destino y se cierra el socket, el bloque finally limpia el stream y el cliente, y encola OnDisconnected. Con autoReconnect activo (por defecto), ConnectionLoop espera reconnectDelay, que vale tres segundos, y vuelve a intentar conectar mientras running siga activo. Al cerrar la aplicación, OnApplicationQuit detiene el ciclo, espera brevemente al hilo y cierra los recursos de red.

La conexión/desconexión de red se distingue de mostrar/ocultar el panel: TerminalToggle alterna la visibilidad del Canvas con la tecla T y ofrece métodos para botones. Su código no llama a TelnetClient para desconectar; por tanto, cerrar visualmente el terminal no necesariamente termina la sesión Telnet.

11.4. Flujo de un comando en el terminal CLI

1. El usuario escribe en TMP_InputField, directamente con teclado físico o con los botones virtuales.
2. Al pulsar Enter, TelnetUI verifica la tecla y llama a SendCommand. El botón virtual Entrar llama al mismo método. Una entrada vacía se representa como una línea vacía.
3. TelnetUI retira saltos de línea finales y llama a TelnetClient.Send. La capa cliente añade CRLF y convierte la cadena a ASCII.
4. TelnetClient escribe los bytes en NetworkStream. Si no hay un cliente conectado, devuelve el mensaje "No conectado" al panel.
5. El servidor interpreta el comando y devuelve bytes. TelnetClient los pasa por la cola hacia TelnetUI.AppendOutput.
6. TelnetUI elimina secuencias de control ANSI, normaliza saltos de línea y programa la actualización de TMP_Text. La vista se desplaza hacia el final para mostrar la respuesta reciente.

El texto del terminal se mantiene en buffers en memoria y se limita con maxOutputChars (10 000 por defecto). ClearOutput limpia esos buffers. No se encontró una rutina que guarde automáticamente el historial CLI en un archivo.

11.5. Teclado virtual e interacción de escritura

La clase KeyboardCLI incluye operaciones básicas para insertar un carácter, borrar el último, insertar un espacio y alternar entre grupos de botones normal/mayúsculas. Para el terminal, TelnetUI.VirtualKeyPress implementa además las acciones Entrar, Retroceso, Espacio, flechas izquierda/derecha y Caps. Caps convierte letras a mayúsculas y los dígitos 1–0 a los símbolos asociados definidos por el script.

El componente TypingArea responde a la entrada y salida de las manos etiquetadas LeftHand y RightHand en el área de escritura; activa o desactiva el modelo de cada mano de escritura. Estas funciones apoyan la interacción VR, mientras que el envío de texto continúa pasando por el campo de entrada y TelnetUI.

11.6. Datos que se guardan y exportación

11.6.1. Configuración

config.json se lee desde la carpeta persistente de la aplicación, pero el código revisado no lo crea ni lo modifica. Para personalizar la IP y los puertos, el archivo debe estar colocado en esa ubicación por el proceso de instalación o por una herramienta externa. La ruta exacta depende de la plataforma; el cliente imprime persistentDataPath en el registro de Unity.

11.6.2. Resultados de prácticas

InitialTestManager agrega líneas a tiempos.txt cuando finalizan grupos de tareas. Las líneas incluyen fecha y hora, tipo de grupo (marcadores A-B-C o movimientos M-RX-RY) y duración en segundos. WireTestManager agrega registros con fecha y hora, el rótulo "Ejercicio de

cableado" y el tiempo empleado. Ambos usan File.AppendAllText, de modo que conservan las entradas previas agregando nuevas líneas al final.

11.6.3. Exportación

En los scripts revisados no aparece una acción de interfaz para exportar datos ni una generación de CSV, Excel o PDF. El archivo tiempos.txt es un registro local de texto que puede localizarse mediante persistentDataPath y copiarse usando un mecanismo externo, pero eso no equivale a una función de exportación integrada en el programa. La salida CLI tampoco se exporta automáticamente.

11.7. Privacidad y seguridad de la comunicación

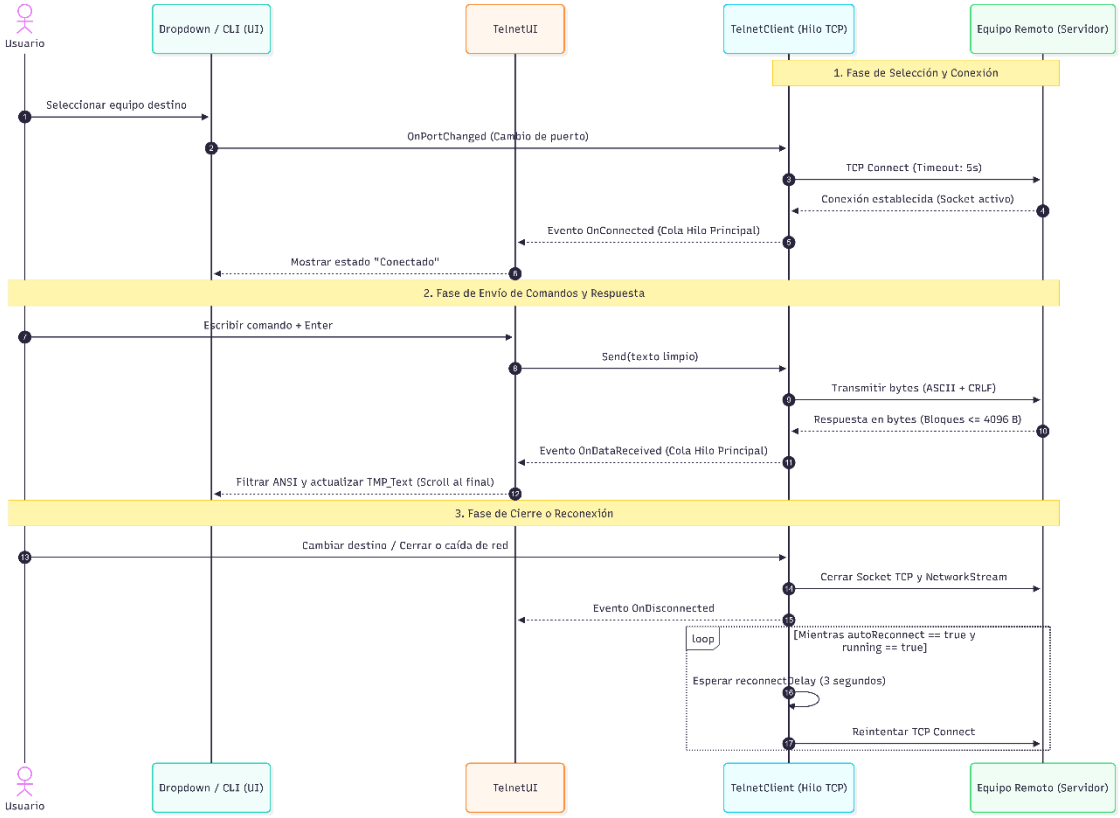
La aplicación envía los comandos como bytes ASCII sobre TCP y recibe la salida del equipo. En TelnetClient no se observa cifrado TLS ni un protocolo SSH, y tampoco se ve una gestión de credenciales propia. Telnet convencional transmite datos sin cifrado; por ello, el contenido podría ser visible para otros participantes de una red no confiable. No se deben introducir contraseñas ni datos sensibles en redes públicas o compartidas.

En el código inspeccionado, la configuración se lee localmente y los tiempos se escriben localmente; no se encontró que estos archivos se envíen a un servicio externo. Esta conclusión cubre los scripts citados, no necesariamente plugins, SDKs, servicios de telemetría o componentes fuera de ese alcance. La aplicación registra la ruta persistente y algunos mensajes de conexión en el log de Unity.

11.8. Diagrama de secuencia de conexión Telnet

El usuario selecciona el destino, el cliente abre el canal TCP, y luego los comandos y respuestas atraviesan las capas mostradas. El panel visualiza eventos, pero el socket pertenece a TelnetClient.

Figura 8. Diagrama de secuencia de la conexión Telnet.



Fuente: Elaboración propia a partir del análisis de los scripts del proyecto Laboratorio Virtual de Redes.